

MCT: Markov Chain Deriver/Generator/Tester Project Specification

=====

1 Core constraints / things to keep in mind

=====

- Channels, not just Kraus operators

A *valid* input is a completely positive trace-preserving (CPTP) map with Kraus form

$$\rho' = \mathcal{E}(\rho) = \sum_k E_k \rho E_k^\dagger \quad (1)$$

and

$$\sum_k E_k^\dagger E_k = I. \quad (2)$$

Code should either enforce or check this normalization when given E_k .

- Basis is part of the specification

You must always carry an explicit ordered basis

$$\mathcal{B} = \{ |b_0\rangle, |b_1\rangle, \dots, |b_{d-1}\rangle \} \quad (3)$$

for the $d = 2^n$ -dimensional Hilbert space. The Markov chain lives on these labels; changing basis changes the effective Markov dynamics.

- Dimensional blow-up and symbolic limits

For n qubits, ρ is $2^n \times 2^n$, and the superoperator is $4^n \times 4^n$. Practical rule:

- “Fully symbolic, many-parameter” mode: realistically only for $n \leq 2$.
- Beyond that, numeric-only or *very* restricted symbolic parameters.
- Distinguish exact quantum evolution vs population Markov chain

Exact evolution is the channel:

$$\rho_{t+1} = \mathcal{E}(\rho_t). \quad (4)$$

The *classical* Markov chain is an *approximate* description of populations in a chosen basis, defined by some rule such as:

- Start from basis projector $\rho_i = |b_i\rangle\langle b_i|$.
- Evolve: $\rho'_i = \mathcal{E}(\rho_i)$.
- Define transition probabilities

$$P_{ji} = \langle b_j | \rho'_i | b_j \rangle. \quad (5)$$

This gives a column-stochastic matrix P when $\mathcal{E}$ is trace-preserving, but it discards coherence information by construction.

- Discrete step vs continuous-time generator

Decide which you are using as the *primary* object:

- Discrete channel per time-step Δt :

$$\rho_{n+1} = \mathcal{E}_{\Delta t}(\rho_n). \quad (6)$$

- Continuous-time Lindbladian generator $\mathcal{L}$:

$$\frac{d\rho}{dt} = \mathcal{L}(\rho), \quad \mathcal{E}_t = e^{t\mathcal{L}}, \quad (7)$$

with $\mathcal{L}$ in Lindblad form

$$\mathcal{L}(\rho) = -\frac{i}{\hbar}[H, \rho] + \sum_j \left(2A_j \rho A_j^\dagger - A_j^\dagger A_j \rho - \rho A_j^\dagger A_j \right). \quad (8)$$

Make the choice explicit in the API, because it affects how you interpret T_1 , T_2 , and per-step probabilities.

- Two backends from day one
- Symbolic backend (e.g. SymPy matrices) for small- n analytic expressions.
- Numeric backend (NumPy / PyTorch) for realistic system sizes and for validating the Markov approximation against full density-matrix evolution.

=====

2 High-level TODO list for the agents

=====

Structure this as the module roadmap. Each bullet is a self-contained work package.

1. Define core data structures

- A `QuantumChannel` object that stores:
- Kraus operators $\{E_k\}$ (symbolic or numeric matrices) in a fixed basis.
- Dimension d and number of qubits n .
- Optionally, the superoperator matrix S acting on vectorized density matrices:

$$|\rho'\rangle\rangle = S |\rho\rangle\rangle. \quad (9)$$

- A `Basis` object:
- Explicit ordered list of computational basis states for n qubits (`["00...0", ...]`).

2. Implement “apply channel to density matrix”

- Core function: `rho_out = apply_channel(channel, rho_in)`

computing

$$\rho_{\text{out}} = \sum_k E_k \rho_{\text{in}} E_k^\dagger. \quad (10)$$

- Should work in both:
- symbolic mode (small systems, returns SymPy expressions), and
- numeric mode (returns NumPy or PyTorch arrays).

3. Implement Markov-chain extraction from a channel

- Function: `P = make_markov_from_channel(channel, basis)`
- Default rule (document this in the code!):
- For each basis state $|b_i\rangle$:

$$\rho_i = |b_i\rangle\langle b_i|, \quad \rho'_i = \mathcal{E}(\rho_i). \quad (11)$$

- Define

$$P_{ji} = \langle b_j | \rho'_i | b_j \rangle. \quad (12)$$

- Verify in code:
- $P_{ji} \geq 0$ and each column sums to 1 (within tolerance).

4. Implement a density-matrix simulator

- Function for *discrete* channels: `rho_t = simulate_density_discrete(rho0, channel, steps)`

repeated application:

$$\rho_{k+1} = \mathcal{E}(\rho_k). \quad (13)$$

- Optional: function for *continuous-time* generator $\mathcal{L}$:
- either via exponentiation $\mathcal{E}_t = e^{t\mathcal{L}}$ for small systems, or numeric ODE integration of

$$\frac{d\rho}{dt} = \mathcal{L}(\rho). \quad (14)$$

5. Cross-check: Markov chain vs full density evolution

- Given P and `simulate_density_discrete`, implement a test function:
- Start from each basis projector $\rho_i = |b_i\rangle\langle b_i|$.
- Evolve with both:
- full channel (density matrix), and
- Markov chain on populations.
- Compare

$$\text{diag}(\rho_k) \quad \text{vs} \quad P^k e_i, \quad (15)$$

where e_i is the i -th standard basis vector.

- Use this to quantify how “classical” the channel looks in a given basis.

6. Build a library of standard test channels Implement constructors (both symbolic and numeric) for:

- Single-qubit amplitude damping (AD): Kraus operators

$$E_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\lambda} \end{pmatrix}, \quad E_1 = \begin{pmatrix} 0 & \sqrt{\lambda} \\ 0 & 0 \end{pmatrix}. \quad (16)$$

- Two-qubit amplitude damping via tensor products

$$K_0 = E_0 \otimes E_0, \quad K_1 = E_0 \otimes E_1, \quad K_2 = E_1 \otimes E_0, \quad K_3 = E_1 \otimes E_1. \quad (17)$$

- Single-qubit pure dephasing (phase damping) with Kraus operators like

$$K_0 = \sqrt{1-p} I, \quad K_1 = \sqrt{p} Z. \quad (18)$$

- Simple depolarizing or Pauli channels.

These give known steady states and known T_1 , T_2 relations for validation.

7. Validation and sanity checks

- CPTP check:
- Verify

$$\sum_k E_k^\dagger E_k \approx I \quad (19)$$

numerically, or exactly in symbolic mode.

- Markov matrix check:
- $P_{ji} \geq 0$ and column sums $\sum_j P_{ji} = 1$.
- Physics sanity:
- AD channel relaxes to ground state; dephasing leaves populations unchanged.
- For AD with rate $\gamma = 1/T_1$, verify population decay

$$\rho_{11}(t) \sim e^{-t/T_1} \quad (20)$$

and coherence decay consistent with chosen T_2 .

8. Document limitations explicitly

- State in the README/API:
- Maximum recommended n for symbolic mode.
- Default basis choice if none is supplied (e.g. computational).
- What Markov-construction rule is used and what information is lost (coherences).

=====

3 Minimal reference set for the agents

=====

To keep them focused, refer to the following resources with indicated page ranges:

1. `qubit_guide.pdf` – **Chapter 9 (Quantum channels)** Use this for the *definitions and structure* of channels and Kraus operators.
 - Pages 170–171: introduction to quantum channels, “everything is secretly unitary”, and the idea that non-unitary evolution arises from unitary evolution + ancilla + partial trace.
 - Page 177: Stinespring dilation and derivation of the operator-sum form

$$\rho \mapsto \rho' = \sum_i E_i \rho E_i^\dagger \quad (21)$$

with $\sum_i E_i^\dagger E_i = I$.

- Page 178: Kraus representation explicitly stated and discussed; good for citing the core equation and completeness relation.
- Page 181: composition of channels (adding ancilla, unitaries, partial trace) and the fact that each piece has its own Kraus representation; this motivates how you’ll implement composition in code.

That’s enough from this book for the purposes of your project.

2. **T1 T2 noise.pdf – amplitude damping example** Use this as a *worked example* of going from single-qubit AD Kraus operators to two-qubit Kraus operators and Bell-state behavior.

- Page 3: explicit AD Kraus operators

$$E_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\lambda} \end{pmatrix}, \quad E_1 = \begin{pmatrix} 0 & \sqrt{\lambda} \\ 0 & 0 \end{pmatrix}, \quad (22)$$

and construction of two-qubit Kraus operators by tensor products.

Agents can use this as a sanity-check case for their multi-qubit symbolic and numeric implementations.

3. **Manenti_Motta-Do_not_share.pdf – Markovian master equations, T_1 , T_2** Use this as the *bridge* between Lindblad master equations, T_1/T_2 , and the effective channels you are approximating.

- Page 300: general Markovian master equation in Lindblad form, with decay rates γ_k and dissipators

$$\mathcal{D}[A_k](\rho) = A_k \rho A_k^\dagger - \frac{1}{2} \{A_k^\dagger A_k, \rho\}, \quad (23)$$

and explicit amplitude-damping example linking $\gamma = 1/T_1$.

- Page 336: summary table including Markovian master equation, amplitude damping, and dephasing, plus the relations defining T_1 and T_ϕ .

This gives enough physics to relate your discrete-step channel parameters $(\lambda_1, \lambda_\phi)$ to continuous-time T_1, T_2 .

4. **Optional / back-pocket: Quantum Noise_ ... Gardiner & Zoller** Only if you want them to see the “canonical” Lindblad form in a more theoretical setting.

- Page 165: statement of the Lindblad form and discussion that it is the general generator of a quantum Markov process.

You can mark this as “for background only; not required for implementation.”

If you want, the next step can be: turn this into a short, agent-facing spec document (with function names and signatures) that you can paste directly into an agentic coding session.